Software Engineering Quality & Static Analysis Report

Academic Clearance & Grievance Automation Engine

Evaluation Reference: Units 5 & 6 (Software Quality, Testing, Metrics & Refactoring)

---

1. Executive Summary

This document records the static code analysis, code smell detection, and unit test results for the Academic Clearance Automation Engine. The system achieves **0 Flake8 static code violations** and **100% pass rate across all automated unit & integration test suites**.

---

2. Static Analysis Audit (Flake8 / PEP 8)

2.1 Configuration

The static analysis is configured via `.flake8` with standard engineering thresholds:

- **Maximum Line Length:** 140 characters
- **Excluded Paths:** `.git`, `__pycache__`, `venv`, `frontend`, `.pytest_cache`
- **Rule Enforcement:** Full PEP 8 naming, unused import pruning (F401), variable ambiguity detection (E741), whitespace integrity (W293).

2.2 Execution Log

```
$ ./venv/bin/flake8 backend/app
# Result: 0 errors returned (Process exited with code 0)
```

---

3. Code Smells: Identification & Remediation

Code Smell	Risk / Anti-Pattern	Applied Architectural Solution
God Class (Large Class)	Concentrating all departmental logic, rules, and approvals in a single 1,500-line manager.	Refactored into **Chain of Responsibility**: Individual `DepartmentHandler` subclasses encapsulate their own verification policies.
Long Method / Cyclomatic Complexity	Deeply nested `if/elif/else` blocks checking departmental clearance order.	Replaced with recursive handler delegation (`handler.handle()`) and discrete `verify_prerequisites()` predicates.
Feature Envy	Handlers directly querying foreign entity internals.	Encapsulated gate evaluation inside `DepartmentHandler.refresh_pipeline_gates()` and model relationship properties.

Code Smell	Risk / Anti-Pattern	Applied Architectural Solution
Ambiguous Identifiers	Variable names such as single letters (`l` , `o`).	Corrected during flake8 audit to descriptive tokens (`log_entry` , `approval_record`).

---

4. Automated Testing & Verification Suite

4.1 Pytest Execution Summary

The test suite validates both behavioral design pattern mechanics and end-to-end REST API workflows.

```
============================= test session starts ==============================
platform darwin -- Python 3.12.5, pytest-9.1.1
rootdir: /Users/raghavbhardwaj/Documents/antigravity/wise-euclid
configfile: pytest.ini
testpaths: backend/tests

backend/tests/test_chain_of_responsibility.py::test_digital_signature_generation PASSED [ 10%]
backend/tests/test_chain_of_responsibility.py::test_gate1_parallel_approvals PASSED      [ 20%]
backend/tests/test_chain_of_responsibility.py::test_gate2_dependency_enforcement PASSED  [ 30%]
backend/tests/test_chain_of_responsibility.py::test_gate3_final_clearance_and_certificate_minting PA
backend/tests/test_clearance_api.py::test_health_check PASSED                            [ 50%]
backend/tests/test_clearance_api.py::test_student_login PASSED                           [ 60%]
backend/tests/test_clearance_api.py::test_staff_login PASSED                             [ 70%]
backend/tests/test_clearance_api.py::test_admin_analytics PASSED                         [ 80%]
backend/tests/test_clearance_api.py::test_public_certificate_verification PASSED         [ 90%]
backend/tests/test_clearance_api.py::test_grievance_submission_and_sla PASSED           [100%]

======================== 10 passed in 1.06s ========================
```

4.2 Test Coverage Highlights

- **Parallel Independence:** Proves Gate 1 departments (Library, Hostel, Sports) operate concurrently without dependency on each other.
- **Sequential Guarding:** Proves Department Labs cannot be approved when Gate 1 departments are pending.
- **Cryptographic Tamper-Evidence:** Confirms HMAC-SHA256 signatures match the exact staff member, department, and clearance request.
- **SLA Breach Engine:** Confirms automatic deadline calculation (Critical: 4h, High: 12h, Medium: 24h, Low: 48h) and auto-escalation upon breach.